

WEB DESIGN

HTML • CSS • JavaScript • PHP • MySQL

Web Design Course By Instructor Eric BYIRINGIRO(0782427392)

Discussion: What Will You Learn?

- What it means to be a Front-End, Back-End, or Full-Stack developer
- Programming languages relevant to web development — HTML, CSS, JavaScript, PHP, SQL
- Frameworks & tools that accelerate professional web development
- How to design, build, and deploy complete web applications
- Best practices in web standards, accessibility, and responsive design

Course Overview

This self-paced course is designed to provide the necessary skills and training for an entry-level position in the field of Web design. The student learns to develop and maintain Web sites for a corporation or one's small business. The class focuses on Web page planning, basic design, layout and construction, and setup and maintenance of a web site.

HTML

CSS

JavaScript

PHP

MySQL

Responsive Design

Forms

DOM

Learning Outcomes

- To understand Web and its relationship with the Internet
- To know who is a Designer, what he/she has to do and different specializations of Web designers
- To describe different tools used in order to design a professional Web site
- To write tags/codes of HTML, CSS and JavaScript, three languages mainly discussed in this lesson
- To develop and to maintain a Web site for a small organization
- To be able to customize the existing web pages and reuse them in our projects

Forms

DOM

Course Contents

- Introducing Web Design
- Hypertext Markup Language (HTML)
 - HTML Text Layout Tags
 - Understanding Hyperlinks
 - Adding Images to Web Pages
 - HTML Tables
 - HTML Forms
- Cascading Style Sheets (CSS)
 - Formatting Fonts and Text with CSS
 - CSS Colors, Padding, Margins & Borders
- Introduction to JavaScript
- PHP & Server-Side Scripting
- MySQL Database Integration

The Process of Interacting with a Website

1 User Input

User enters URL or clicks link;
browser prepares request

2 HTTP Request

Browser sends HTTP/HTTPS
request — may include form
data or API call

3 Server Processing

Server executes back-end
code (PHP, Node.js, Python);
queries database

4 Server Response

Server sends back HTML, CSS,
JS, or JSON data

5 Browser Rendering

Browser converts raw code
into the visual, interactive
webpage you see on your
screen

6 Ongoing Interaction

User continues interacting;
each action may repeat the
cycle

Front-End vs Back-End Development

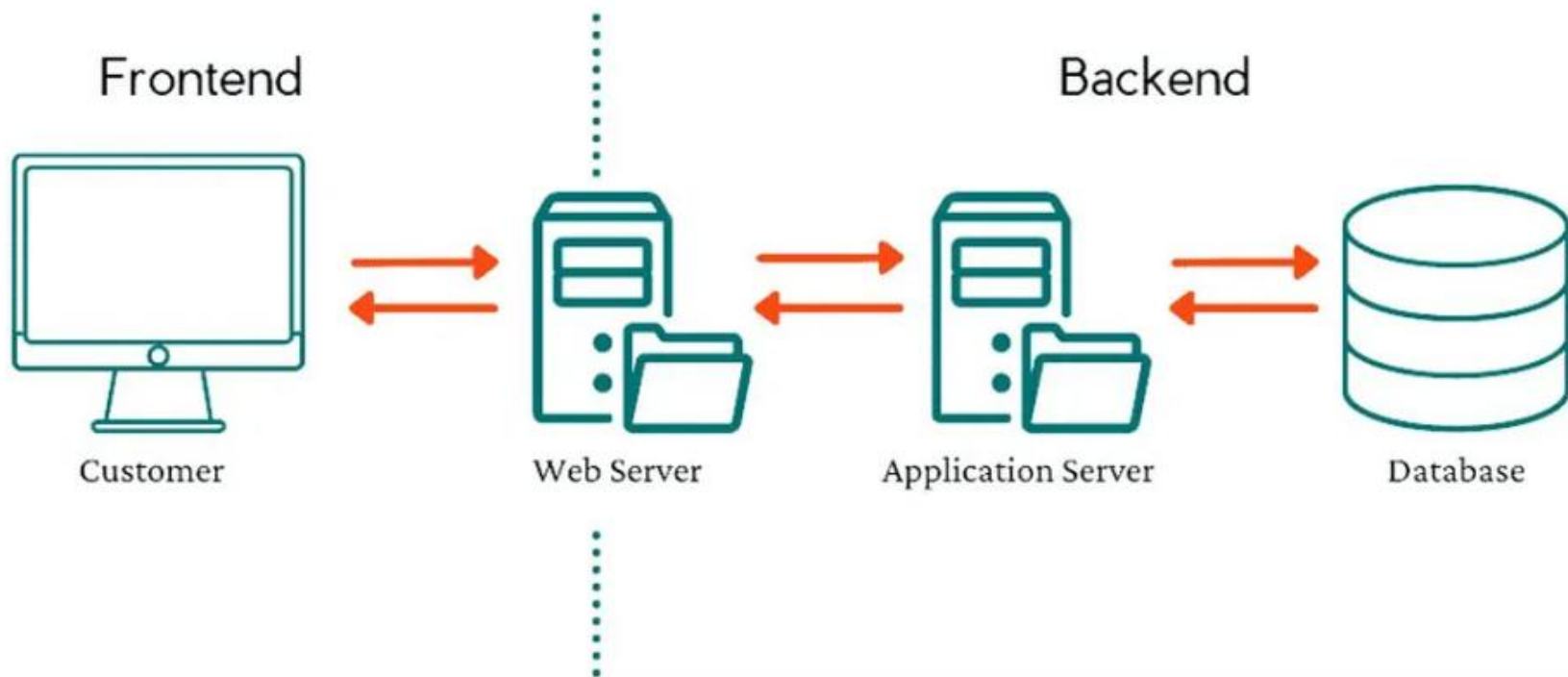
Front-End

- Visual part users interact with
- Languages: HTML, CSS, JavaScript
- Framework: Bootstrap
 - Bootstrap is a frontend toolkit consisting of pre-styled CSS elements and utility classes used for layout and design.
- Tools: Figma, Adobe XD, Sketch
 - Figma – A tool for UI/UX design, prototyping, and vector graphics.
 - Adobe XD – A vector-based application for designing and prototyping user experiences.
 - Sketch – A macOS-based vector graphics editor focused on UI/UX design.
- Focus: UI/UX, responsive design

Back-End

- Part Responsible for executing operations, managing data, and communicating with databases or APIs
- Server, database & logic
- Languages: Node.js, Python, PHP, Java
- Databases: MySQL, PostgreSQL, MongoDB
- Tasks: APIs, authentication, data management
- Handles requests from the front-end
- Full-stack: expertise in both sides

How to Connect Frontend and Backend Applications





HTML

Hyper Text Markup Language — The Structure of the Web

1

What is HTML?

- HTML = HyperText Markup Language — the standard language for creating web pages
- Technically a markup language, not a programming language — it describes structure and content
- HyperText: allows navigation between pages via clickable hyperlinks
- Markup: HTML tags instruct the browser how to display content
- HTML files are plain text files parsed by web browsers to render visual pages
- Works alongside CSS (styling) and JavaScript (behaviour) to build modern websites

HTML Versions — A Brief History

HTML 1.0

Bare-bones; no tables, fonts or background;
Lynx/Mosaic only

HTML 3.2

Tables, complex equations; W3C published by Tim Berners-Lee

XHTML

XML-based; stricter syntax; ensured code interoperability

HTML 2.0

Forms, text boxes, page backgrounds;
W3C formed

HTML 4.01

Style sheets, scripting, multimedia;
content/presentation separation

HTML 5

Semantic elements, audio/video, Canvas, offline support — current standard

1989–94

1995

1997

1997–99

2000

2014

HTML Editors

- A good HTML editor keeps your code clean, organised, and error-free
- Editors auto-close tags — reducing bugs and saving typing time
- Most modern editors include a WYSIWYG preview feature

❏ Popular Free Editors:

- Notepad / Notepad++ — lightweight plain text editing
- **VS Code (Visual Studio Code)** — most popular, rich extensions, IntelliSense
 - Install **Live Server** by **Ritwick Dey**
 - Open your HTML file. **Right-click** inside the HTML file and select "Open with Live Server" or click "Go Live" in the bottom-right corner.
- JSFiddle — browser-based; great for quick experiments
- HTML-Kit — dedicated HTML editor with built-in browser preview

Basic Document Structure

```
<!DOCTYPE html>           <!-- Declares HTML5 document type -->
<html lang="en">           <!-- Root element — wraps everything -->

  <head>                     <!-- Metadata (not visible on page) -->
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My Web Page</title>  <!-- Tab title in browser -->
    <link rel="stylesheet" href="styles.css">
  </head>

  <body>                     <!-- Visible page content goes here -->
    <h1>Hello, World!</h1>
    <p>Welcome to my first web page.</p>
  </body>

</html>
```

Practical Exercise1

- Install Visual Studio Code (VS Code) on your computer and launch the application. Create a new folder named **HTML_Exercise** and inside it create a file called **index.html**. Using VS Code, write a complete HTML document and include the following content in the body section: an H1 heading titled **"Viewport"** followed by a paragraph explaining that “the viewport element gives the browser instructions on how to control a page's dimensions and scaling”. Add an H2 heading titled **"How it works"** followed by a paragraph describing how “the **width=device-width** setting allows the page width to match the screen width of the device.” Then add an H3 heading titled **"Initial Scale"** followed by a paragraph explaining that “**initial-scale=1.0** sets the initial zoom level when the page is first loaded by the browser.” Save the file and run it in a

Tags

❏ A **tag** is a piece of code used to build the structure of a website. It acts as a command or instruction that tells a web browser exactly how to format, organize, and display text, images, or media on the screen

- Keywords in angle brackets: `<tag>`
- Come in pairs: `<p> ... </p>`
- Opening tag: `<body>`
- Closing tag: `</body>`
- Empty/void tags: These tags are completely unique because they cannot contain any inner text content and do not require a closing tag.
- `<br>`: Line break, `<hr>`: Horizontal rule, `<img>`: image
- Browsers render tags, not display them

Elements

❑ People often confuse "tags" and "elements," but they are technically different. A tag is just the markup code (`<p>` or `</p>`), whereas an element refers to the entire bundle: the opening tag, everything written inside it, and the closing tag altogether.

- `<tagname> Content </tagname>`
- Everything from start to end tag
- Empty elements have no content
- Elements can be nested
- Example: `<p>Hello <strong>World</strong></p>`

Attributes

- ❑ An **attribute** is a special modifier word placed inside an HTML opening tag that provides extra information, configurations, or settings for that element.
 - Provide extra info about elements
 - Written in the opening tag only
 - Format: name="value"
 - href, src, style, class, id ...
 - HTML5 attributes are case-insensitive

Common Attribute Examples

```
<!-- Hyperlink - href attribute -->
```

```
<a href="https://www.w3schools.com">Visit W3Schools</a>
```

```
<!-- Image - src and alt attributes -->
```

```

```

```
<!-- Inline style - style attribute -->
```

```
<p style="color: red; font-size: 18px;">This paragraph is red and large.</p>
```

```
<!-- Heading with an id -->
```

```
<h1 id="main-title">Welcome to My Website</h1>
```

```
<!-- Paragraph with a class -->
```

```
<p class="intro-text">This paragraph can be styled with CSS using .intro-text</p>
```

Practical Exercise2: Enhancing a Web Page with Attributes, Images, and Hyperlinks

- Open the HTML file created in Exercise 1 and modify it by adding a **bgcolor** attribute to the <body> tag to change the background color of the webpage. Below the **Initial Scale** paragraph, insert an image of a Bible and set an alternative text message of "**holly bible**". The image should have a width of **100 pixels**.
- Below the image, create a paragraph that displays the text "**Hello Believers in Jesus Christ**", with the words "**Jesus Christ**" appearing in bold. Continue the paragraph with the text "**for more information about our church visit Adventist**", where the word "**Adventist**" functions as a hyperlink that directs users to the Adventist Church website at <https://gc.adventist.org/>.
- Save the file and view it in a web browser to verify that the background color, image, bold text, and hyperlink are displayed correctly.

HTML Formatting Elements(Text Formatting)

`<b>`

Bold text — visual only

`<strong>`

Important text — semantic bold

`<i>`

Italic text — visual only

`<em>`

Emphasized text — semantic italic

`<mark>`

Highlighted / marked text

`<small>`

Smaller text

`<del>`

Deleted / strikethrough text

`<ins>`

Inserted / underlined text

`<sub>`

Subscript text (e.g. H₂O)

`<sup>`

Superscript text (e.g. x²)

- Create a single HTML page that demonstrates the use of inline text formatting tags: `<b>`, `<i>`, `<mark>`, `<del>`, `<sub>`, `<strong>`, `<em>`, `<small>`, `<ins>`, and `<sup>`. The page should follow a standard HTML structure with `<!DOCTYPE html>`, `<html>`, `<head>`, and `<body>`. In the body, write a title for the page such as “Scientific Notation and Chemistry Notes” and format it using `<strong>` and `<em>` to make it both bold and emphasized. Then, write a paragraph about chemistry or ICT concepts where you apply different formatting tags within the text: use `<b>` to highlight an important term, `<i>` to italicize a scientific or foreign word, `<mark>` to highlight a key safety instruction or important concept, and `<small>` to include a short note or disclaimer at the end of the paragraph.
- Include another paragraph that presents scientific expressions or formulas, making use of `<sub>` for subscripts (for example in chemical formulas like H₂O or CO₂) and `<sup>` for superscripts (for example powers like x² or 10³). After that, include a sentence that demonstrates correction of information by using `<del>` to show incorrect text and `<ins>` to show the corrected version, such as correcting a chemical formula. Finally, add a concluding statement that emphasizes the importance of the topic using both `<em>` and `<strong>` to show strong emphasis and importance. The overall content should be meaningful, well-structured, and demonstrate correct usage of all required HTML formatting tags within a coherent scientific or academic context.

Thank You!

HTML · CSS · JavaScript · PHP · MySQL

Keep building. Keep learning. 